

Student information Sender

Course Code - SE-409

Answer to the question no - 2 @

In my project I used Java Collection to store and manage data efficiently. The main collection I used was hash map and sometime Array / Array list for storing multiple records.

→ why hash map was chosen?

⇒ I used →

`HashMap <Integer, Student>`

because →

- i) It stores data in key value / pairs
- ii) Student ID can be used as a unique key.
- iii) Searching data become very fast.

iv) It helps to find duplicate student ID.

Example →

HashMap <Integer, Student> students = new

HashMap <> ();

here →

Key → student ID

Value → student object

→ why Array or ArrayList chosen?

⇒ Array or ArrayList used to:

Store multiple student records sequentially.

Display all results easily.

Example →

ArrayList <Student> list =

new ArrayList <> ();

→ How collection helped?

⇒ Java collection help the program to →

- Add student information dynamically.
- Search student records quickly.
- Update or delete data easily.
- Organize result efficiently.

So, using collection make the program more flexible and easier to maintain.

Ans to the question no- 2(b)

Socket communication was used to establish communication between the client and the server.

The server waits for connection request and the client connects to the server using an IP address and port number. After connection both sides can send and receive data.

→ Roles of server and Server Socket.

Server socket →

Used to on the server side.

- Listen for incoming client request.
- Create a communication channel.
- Accept client connection.

Example →

server socket ← serverSocket =

new ServerSocket(5000);

Socket →

Socket is used to on both client
and server side.

→ For sending data

→ For receiving data

→ Maintain communication between
client and server.

Example →

Socket socket = new Socket("localhost", 5000);

Answer to the question no - 2C

Steps to send and receive data using socket →

Step: 01: → Create ServerSocket on Server.

The ~~Server~~ server create server socket using a port number.

Ex → `ServerSocket server = new ServerSocket(5000);`

Step: 02 → Accept client connection.

The server waits for the client connection.

Ex → `Socket socket = server.accept();`

Step:03 → Create socket on client.

The client connects to the server using IP address and port number.

Step:04 → Create input and output stream.

EX →

DataInputStream dis = new

DataInputStream (Socket.getInputStream());

DataOutputStream dos = new DataOutputStream (Socket.getOutputStream());

Step:05 → Send Data

The client sends data using output stream.

Example →

dos.writeUTF ("Hello Server");

Step: 6 → Receive data

The receiver reads data using input stream.

Example →

Java `String msg = dis.readUTF();`

Step: 7 → Close Connection

Conclusion → Java collection helped

(manage and organize student data efficiently while socket programming enabled communication between client and server.

(Source: All Java String)